Embedded C Interview Preparation – Questions & Answers

1. Memory Layout

Text Segment: Stores program code, typically in Flash, read-only.

Data Segment: Stores initialized global and static variables.

BSS Segment: Stores uninitialized global and static variables, auto-zeroed.

Heap: Used for dynamic memory allocation; often avoided in embedded systems.

Stack: Stores local variables and function call frames; limited in size.

2. Storage Classes & Keywords

static: Provides internal linkage and persistent lifetime.

extern: Used to reference variables/functions defined in other files.

volatile: Prevents compiler optimization for variables modified by ISR or hardware.

register: Suggests storing variable in CPU register (mostly ignored by modern compilers).

3. Pointers

Pointer: Stores address of another variable.

Pointer arithmetic: Used to traverse arrays and buffers.

Pointer-to-pointer: Used to modify pointer values in functions.

Function pointer: Used for callbacks and dynamic function calls.

Void pointer: Generic pointer requiring type casting before dereferencing.

4. Bitwise Operations

Set/Clear/Toggle bits using bitwise operators.

Masking: Used to extract or modify specific bits.

Packing signals: Common in CAN/LIN communication.

Shifting: Used for fast multiply/divide and bit positioning.

Endianness: Determines byte order in memory.

5. Structures, Unions & Bit-fields

Structure: Groups related variables with independent memory.

Union: Shares memory among members; size equals largest member.

Bit-fields: Allow access to individual bits; useful for hardware registers.

Alignment & padding affect sizeof(struct).

Packed structures prevent padding for protocol mapping.

6. Compilation Stages

Preprocessing: Expands macros and includes headers.

Compilation: Converts C code to intermediate representation.

Assembly: Converts code to machine instructions (.o).

Linking: Resolves symbols and produces final ELF/BIN.

Function handling includes symbol table entry and stack frame creation.

7. RTOS Basics

Task creation and scheduling are core RTOS concepts.

Mutex, semaphore, and event groups provide synchronization.

Priority inversion handled using priority inheritance.

Tick interrupt drives task switching and delays.

ISR context differs from task context.

8. Peripheral Basics

GPIO, UART, I2C, SPI are common MCU peripherals.

Polling vs interrupt-based communication.

Timers, watchdog, and PWM are essential for real-time control.

Memory-mapped I/O allows register access via pointers.

Interrupt flow uses vector tables and ISRs.

9. Linux Basics (Embedded)

Process vs thread differences.

Context switching and scheduling basics.

Virtual memory and system calls.

Fork/exec model and file descriptors.

Blocking vs non-blocking I/O and poll/select.

10. Data Structures (Embedded-Oriented)

Array: Contiguous memory, fast access.

String: Character array with null terminator.

Linked list: Dynamic structure for flexible data.

Stack: LIFO, often array-based.

Queue: FIFO, often implemented as circular buffer.

Interview Tip

Focus on practical embedded concepts rather than algorithm-heavy problems. Understand memory, pointers, RTOS, peripherals, and compiler behavior deeply.